Deploy Hospitality App to Kubernetes

Course Name: Devops_Foundation

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	KAIZAR YUSUF	EN22CS301475
2.	MADHUR KUKREJA	EN22CS301561
3.	JAYESH VERMA	EN22CS301466
4.	KARTAVYA DONGRE	EN22CS301485
5.	KHUSHI SINGH	EN22CS301506

Group Name: G08D5

Project Number: DO_08

Industry Mentor Name: Mr. VAIBHAV

University Mentor Name: Dr. RITESH JOSHI

Academic Year: 2025-2026

Table Of Content

1. **Problem Statement**
2. **Project Objectives**
3. **Scope of the Project**
4. **Key features**
5. **Overall Architecture / Workflow**
6. **Tools & Technologies Used**
7. **Results & Output**
8. **Conclusion**
9. **Future Scope & Enhancements**

1. Problem Statement

In the modern hospitality industry, web applications play a crucial role in managing customer interactions such as room browsing, booking, and inquiries. These applications are expected to be highly available, scalable, and reliable to ensure a smooth user experience. However, traditional deployment methods often rely on manual processes and static infrastructure, which can lead to deployment errors, downtime during updates, and limited scalability.

Such systems struggle to handle varying user traffic efficiently, especially during peak usage, and lack the flexibility to quickly adapt to changing requirements. Additionally, the absence of automation slows down the development and deployment lifecycle, making it difficult to deliver updates and new features in a timely and consistent manner.

To overcome these challenges, there is a need for an automated, cloud-based deployment solution that incorporates modern DevOps practices. This solution should ensure:

- Continuous Integration and Continuous Deployment (CI/CD) for faster and automated releases
- High availability and scalability to handle dynamic user traffic
- Efficient resource utilization through containerization and orchestration
- Reliable and consistent deployment processes with minimal downtime

This project focuses on implementing such a system using containerization and Kubernetes, providing a scalable and efficient deployment approach for hospitality applications.

2. Project Objectives

The main objectives of this project are:

- To develop a hotel website (Grand Azure Hotel) using Flask
- To implement a complete DevOps pipeline from development to deployment
- To automate the build and deployment process using CI/CD (GitHub Actions)
- To containerize the application using Docker
- To deploy and manage containers using Kubernetes (AWS EKS)
- To ensure high availability using multiple replicas and load balancing
- To use AWS cloud services for scalable and production-ready infrastructure

3. Scope of the Project

This project covers:

- Development of a multi-page hospitality web application
- Implementation of containerization and orchestration
- Deployment on AWS cloud infrastructure
- Automation using CI/CD pipeline
- Use of Kubernetes for scaling and management

Limitations / Out of Scope:

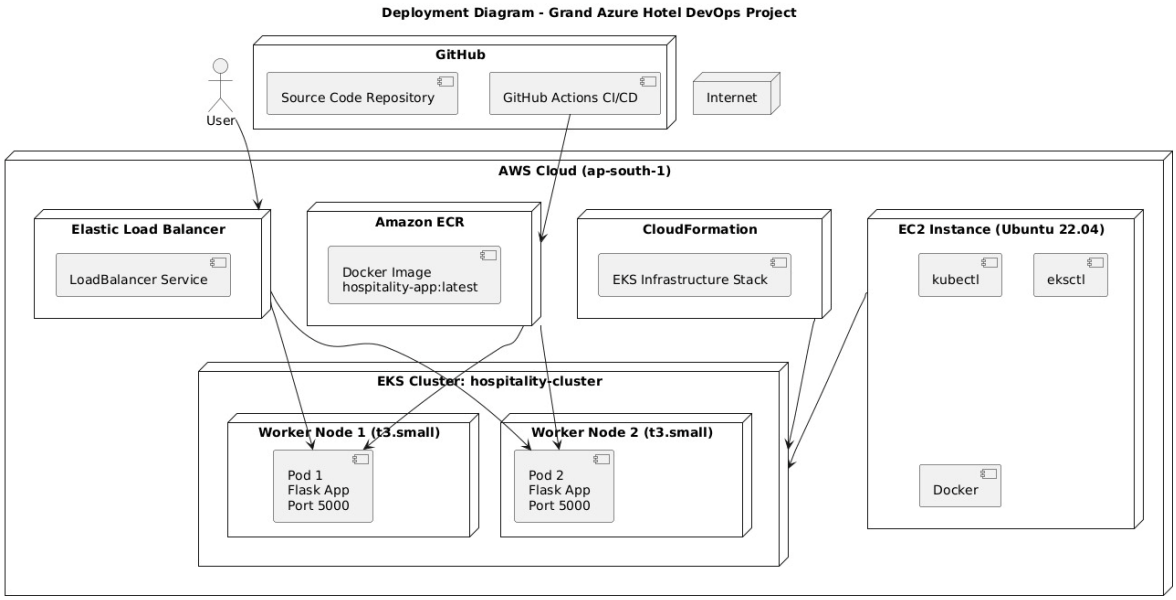
- Advanced security configurations (WAF, HTTPS, etc.)
- Payment gateway integration
- Database integration (if not implemented)
- Monitoring tools like Prometheus/Grafana

4. Key Features

- Automated CI/CD pipeline using GitHub Actions
- Docker-based containerized application

- Kubernetes-based deployment with auto-scaling capability
- High availability using multiple pod replicas
- Load-balanced application using AWS ELB
- Cloud-based infrastructure using AWS services
- Version-controlled code using GitHub

5. Overall Architecture and Workflow



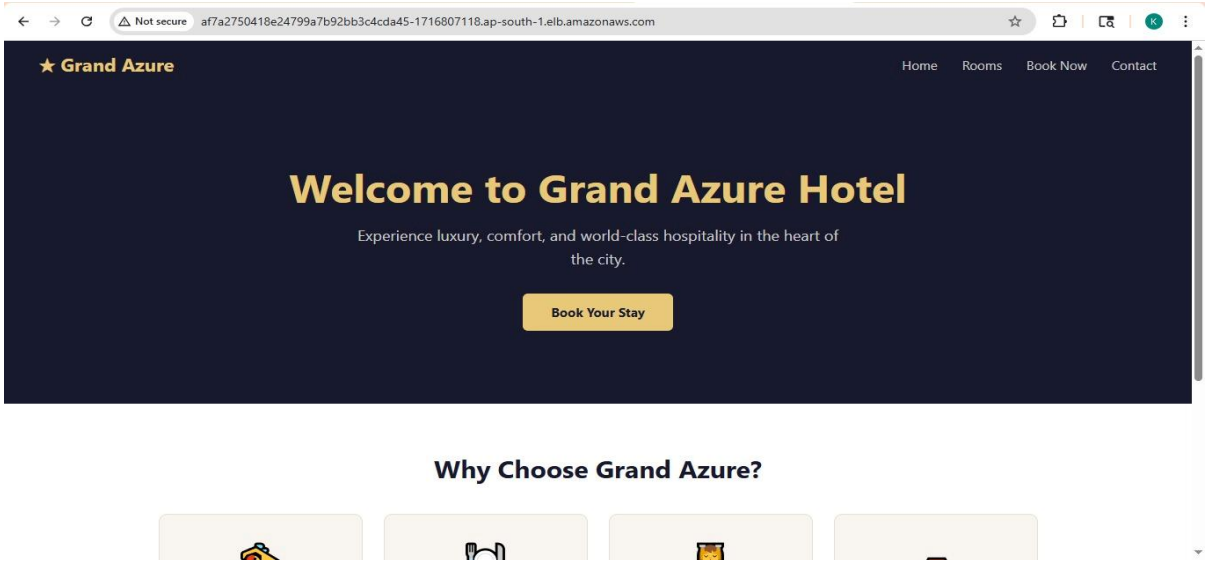
Step-by-step Workflow:

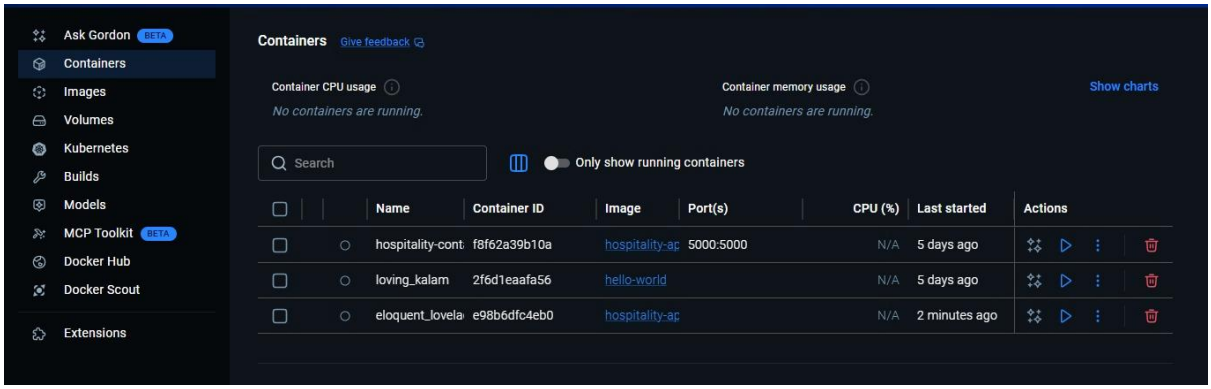
1. Developer writes code on local system (Flask application)
2. Code is pushed to GitHub repository
3. GitHub Actions workflow is triggered on push
4. Docker image is built using Dockerfile
5. Image is pushed to AWS ECR
6. Kubernetes pulls the image from ECR
7. Application is deployed on AWS EKS cluster
8. Multiple pods (replicas) are created for availability
9. LoadBalancer service exposes the application
10. Users access the application via external ELB URL

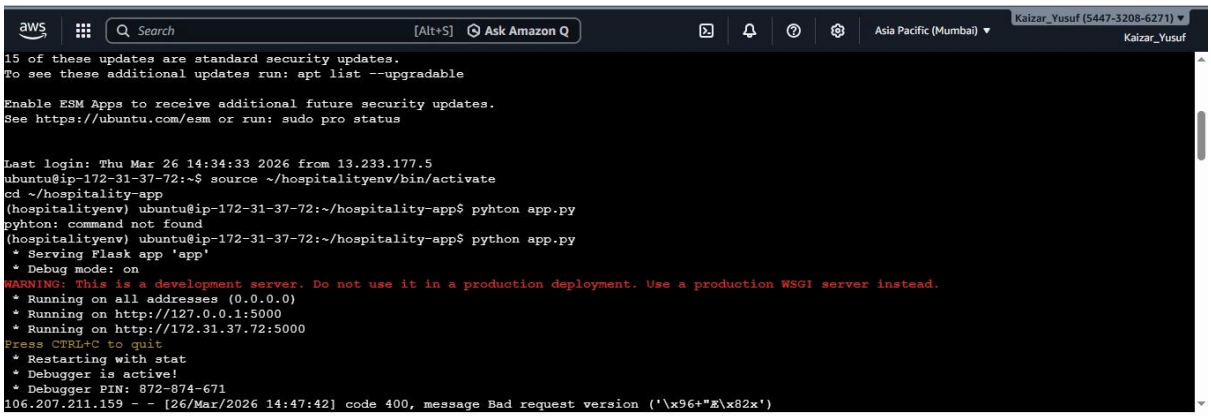
6. Tools & Technologies Used

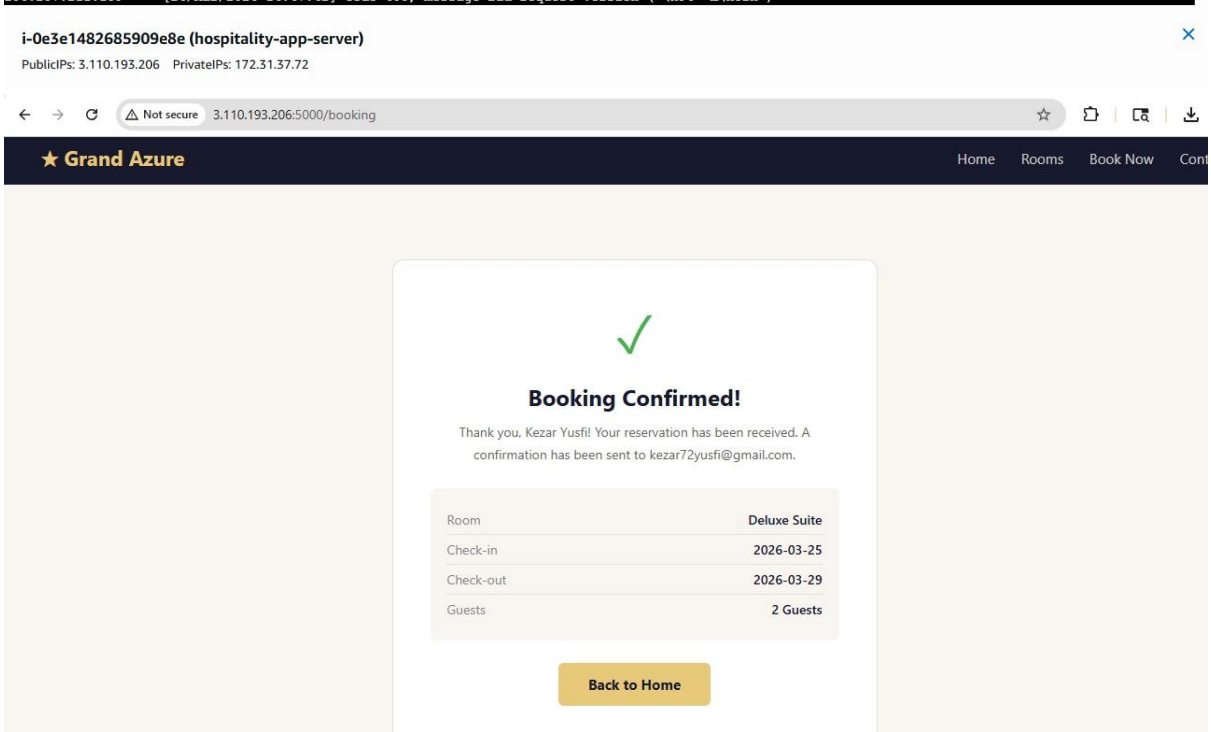
Category	Tools/Technologies
Programming	Python (Flask)
Version Control	Git, GitHub
CI/CD	GitHub Actions
Containerization	Docker
Container Registry	AWS ECR
Orchestration	Kubernetes (AWS EKS)
Cloud Platform	AWS (EC2, EKS, ECR, ELB, IAM, CloudFormation)
OS	Ubuntu 22.04
Infrastructure Automation	eksctl (CloudFormation backend)

7. Result and Output




Ask Gordon BETA
Containers
Images
Volumes
Kubernetes
Builds
Models
MCP Toolkit BETA
Docker Hub
Docker Scout
Extensions
Containers Give feedback
Container CPU usage
No containers are running.
Container memory usage
No containers are running.
Show charts
Search
Only show running containers
Name
Container ID
Image
Port(s)
CPU (%)
Last started
Actions
hospitality-cont f8f62a39b10a hospitality-ap 5000:5000 N/A 5 days ago
loving_kalam 2f6d1eaafa56 hello-world N/A 5 days ago
eloquent_lovela e98b6dfc4eb0 hospitality-ap N/A 2 minutes ago


aws
Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Kaizar_Yusuf (5447-3208-6271)
Kaizar_Yusuf
15 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
Last login: Thu Mar 26 14:34:33 2026 from 13.233.177.5
ubuntu@ip-172-31-37-72:~$ source ~/hospitalityenv/bin/activate
cd ~/hospitality-app
(hospitalityenv) ubuntu@ip-172-31-37-72:~/hospitality-app$ pyhton app.py
pyhton: command not found
(hospitalityenv) ubuntu@ip-172-31-37-72:~/hospitality-app$ python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.37.72:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 872-874-671
106.207.211.159 - - [26/Mar/2026 14:47:42] code 400, message Bad request version ('\x96+"Æ\x82x')


i-0e3e1482685909e8e (hospitality-app-server)
PublicIPs: 3.110.193.206 PrivateIPs: 172.31.37.72
Not secure
3.110.193.206:5000/booking
Grand Azure
Home
Rooms
Book Now
Booking Confirmed!
Thank you, Kezar Yusfi! Your reservation has been received. A confirmation has been sent to kezar72yusfi@gmail.com.
Room Deluxe Suite
Check-in 2026-03-25
Check-out 2026-03-29
Guests 2 Guests
Back to Home

8. Conclusion

- The project "Deploy Hospitality App to Kubernetes" successfully demonstrates the implementation of a complete DevOps pipeline for deploying a real-world web application. A hospitality website, *Grand Azure Hotel*, was developed using Flask and deployed on AWS using modern DevOps practices.
- Through this project, key concepts such as containerization (Docker), continuous integration and delivery (GitHub Actions), and container orchestration (Kubernetes on AWS EKS) were effectively applied. The automated pipeline ensures that any code change is seamlessly built, tested, and deployed without manual intervention.
- The use of Kubernetes with multiple replicas and a LoadBalancer service ensures high availability, scalability, and fault tolerance. Additionally, leveraging AWS services like ECR, EKS, and EC2 provides a cloud-native and production-ready deployment environment.
- Overall, this project highlights how DevOps practices can significantly improve deployment efficiency, reduce human errors, and enable scalable application management in real-world scenarios.

9. Future Scope and Enhancements

The project "Deploy Hospitality App to Kubernetes" provides a strong foundation for cloud-native application deployment. It can be further extended and improved in the following ways:

1. Scalability & Performance Enhancements

- Implement Kubernetes Horizontal Pod Autoscaler (HPA) to automatically scale pods based on CPU or traffic.
- Optimize resource usage using resource requests and limits.
- Introduce caching mechanisms (Redis) to improve response time.

2. Monitoring & Logging

- Integrate Prometheus and Grafana for real-time monitoring of application and cluster performance.

- Use ELK Stack (Elasticsearch, Logstash, Kibana) for centralized logging and debugging.

3. Security Improvements

- Enable HTTPS using AWS Certificate Manager (ACM) and Ingress Controller.
- Use IAM roles for service accounts (IRSA) instead of static credentials.
- Implement Kubernetes Secrets / AWS Secrets Manager for secure data handling.
- Add network policies to restrict unauthorized access.

4. Database Integration

- Integrate a backend database such as:
  - AWS RDS (MySQL/PostgreSQL)
  - AWS DynamoDB
- Store user bookings, customer data, and transaction history.

5. CI/CD Pipeline Enhancements

- Add automated testing (unit & integration tests) in the pipeline.
- Implement staging and production environments.
- Use blue-green or canary deployments for zero-downtime releases.

6. Infrastructure Improvements

- Use Terraform for better Infrastructure as Code (IaC) management.
- Automate full infrastructure provisioning and teardown.

7. Application-Level Enhancements

- Add user authentication and authorization system.
- Implement payment gateway integration.
- Improve UI/UX with modern frontend frameworks (React/Angular).